

Project 1: Virtual Machine and Linux

Due by **September 06, 2024 11:59 pm**

Summary

The first project is a warm-up exercise to help you become familiar with the virtual machine (VM) and Linux software that you will use extensively for your kernel development throughout this semester. In this exercise, you need to create a new VM, install a Linux-based OS on the VM, and install the latest kernel in this Linux system.

Description

Step 1: Create a new VM in VirtualBox.

We will use VMs extensively throughout the projects of this course. They make our lives much easier as kernel developers. We can conveniently test our new kernels without crashing the physical machines and take VM snapshots to save the progress of our work. In addition to completing this step as required in the project, I encourage you to play with your VM and get familiar with these useful features.

Notes:

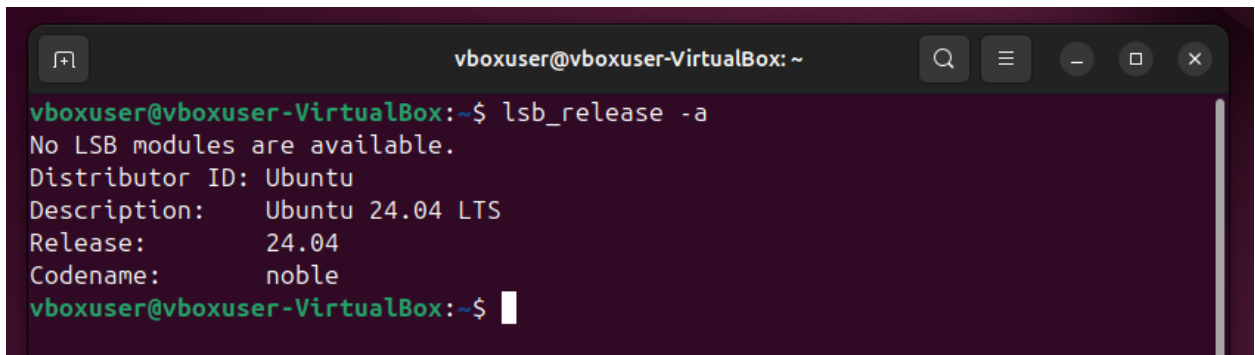
- We will work on a 64-bit kernel in our projects, so make sure your VM is *64-bit* too.
- Kernel compilation is time-consuming, but it can be accelerated by using multiple CPUs to compile it in parallel, so give your VM as many cores as your physical machine has.
- Give your VM more than **50-75GB** of the virtual disk because building a new kernel requires a lot of space. But use dynamic allocation so that its actual storage usage grows as needed.
- Download VirtualBox: <https://www.virtualbox.org/wiki/Downloads>
- VirtualBox User Manual: <https://www.virtualbox.org/manual/>
- If you use a Mac with ARM-based CPUs such as M1, M2, and M3, we suggest you use UTM, instead of VirtualBox, to create your VM, because VirtualBox has known issues with supporting this type of system. **Reference:** <https://mac.getutm.app/>

Step 2: Install Ubuntu 24.04 on your new VM.

Ubuntu is one of the major GNU/Linux distributions and it is quite user-friendly. In this step, you will install Ubuntu (version 24.04 LTS) on your new VM which will be used for all the projects of this course. (LTS means that this version will be supported for a long term. 24.04 is the latest LTS version of Ubuntu, released on April 25, 2024).

Run the following command, `lsb_release -a`, in a terminal to check the Ubuntu version and verify you are using **Ubuntu 24.04 LTS**.

Sample Output:

A terminal window titled 'vboxuser@vboxuser-VirtualBox: ~' with standard window controls. The terminal shows the command 'lsb_release -a' being executed. The output is as follows:

```
vboxuser@vboxuser-VirtualBox:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04 LTS
Release:        24.04
Codename:       noble
vboxuser@vboxuser-VirtualBox:~$
```

Notes:

- Download Ubuntu
 - x86: [Download Ubuntu Desktop | Download | Ubuntu](#)
 - ARM: [Download ARM Ubuntu](#)
- Installation tutorial :
 - x86:
<https://ubuntu.com/tutorials/how-to-run-ubuntu-desktop-on-a-virtual-machine-using-virtualbox#1-overview>
 - ARM: <https://docs.getutm.app/guides/ubuntu/>

Step 3: Compile and install a new kernel on your new Ubuntu

In this step, you will upgrade the kernel in your VM to the latest stable version which will be used as the basis for all your kernel development this semester. The below steps should be done inside the VM.

Basic instructions:Prepare the environment.

1. Download the long-term stable kernel's (6.10) source code from [linux-6.10.tar.gz](https://www.kernel.org/pub/linux/kernel/v6.x/linux-6.10.tar.gz)
2. Uncompress the file into a folder using `tar -xvf linux-6.10.tar.gz`.
3. Install the necessary dependencies for kernel compilation such as **git**, **fakerooroot**, **build-essential**, **ncurses-dev**, **xz-utils**, **libssl-dev**, **bc**, **flex**, **libelf-dev**, **bison**. *Hint:* You can use apt-get to install these packages. Refers to this [howto](#).

Compile the kernel.

4. Configure the kernel. We will start with the current kernel's config file which can be found at `/boot/config-$(uname -r)`. Copy it to the linux source code folder. Name this file as `“.config”`. Use `make oldconfig` to start the configuration process. Press enter to accept all the default options.
5. Use `make menuconfig` to make the following kernel configuration changes and save your changes when prompted.
6. For grading purposes, change the new kernel's version name to include your name. For example, if your name is Linus Torvalds, your local version string should be `“CSE330Fall2024LinusTorvalds”`. You can find this setting in **“General setup”** -> **“Local version”**
7. Check that the **“Symmetric multi-processing support”** option is enabled to allow your kernel to use the multiple cores that your VM has.
8. Disable the configuration that embeds trusted security keys directly into the kernel image, which can be used to verify kernel modules before loading them. We don't need this feature, so we remove it. In menuconfig, you can find it in *Cryptographic API* -> *Certificates for signature checking* -> *“Additional X.509 keys for default system keyring”* and *“X.509 certificates to be preloaded into the system blacklist keyring”*. Uncheck both of them and save the changes.
9. Build the kernel by executing the command `make -j4` (this may take a long time, a few hours, when you do it for the first time). Press enter to accept all the default options. The -j flag enables parallel compilation to speed up the process. The number after -j specifies how many threads to use for parallel compilation. You should set it based on the number of CPUs that your VM has.
Hint: you can check if your compilation is successful by the command `make after above make -j4`. It should not report any errors.
10. Build the kernel modules by executing the command `make modules -j4`

Install the kernel. Now we have successfully compiled the new kernel and its modules. Next, we install them to the default system folders so they can be used to boot up the

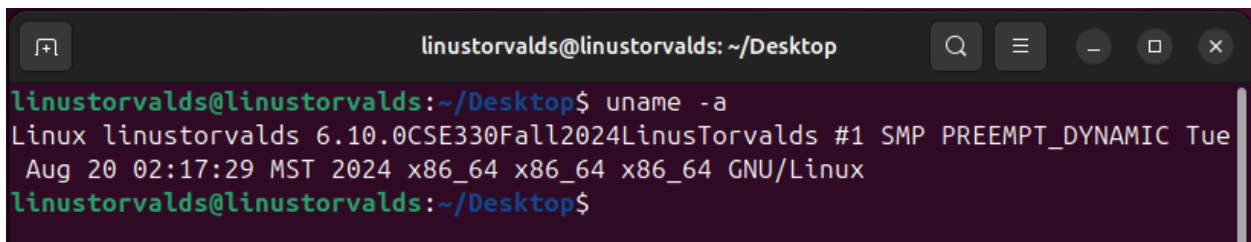
system. Access to these require admin privileges, so you need to use `sudo` to execute the following commands.

11. Install the new kernel modules by executing the command `make modules_install -j4`.
12. Install the new kernel image by executing the command `make install -j4`.
13. Configure the GRUB boot-loader so it can load the new kernel image that you just built. By running the `update-grub` command, GRUB should find all the bootable kernels and add them to the GRUB menu. Restart Linux and select the new kernel's entry from the GRUB menu list. By default the GRUB menu will not be shown; to see it, in `/etc/default/grub` change `GRUB_TIMEOUT=-1` (or long press shift when reboot) and run `update-grub` again; then when you reboot, you should see the menu.

Reboot the VM.

14. Upon next login, check the current running kernel version using `uname -a` and verify that you are using your new kernel.

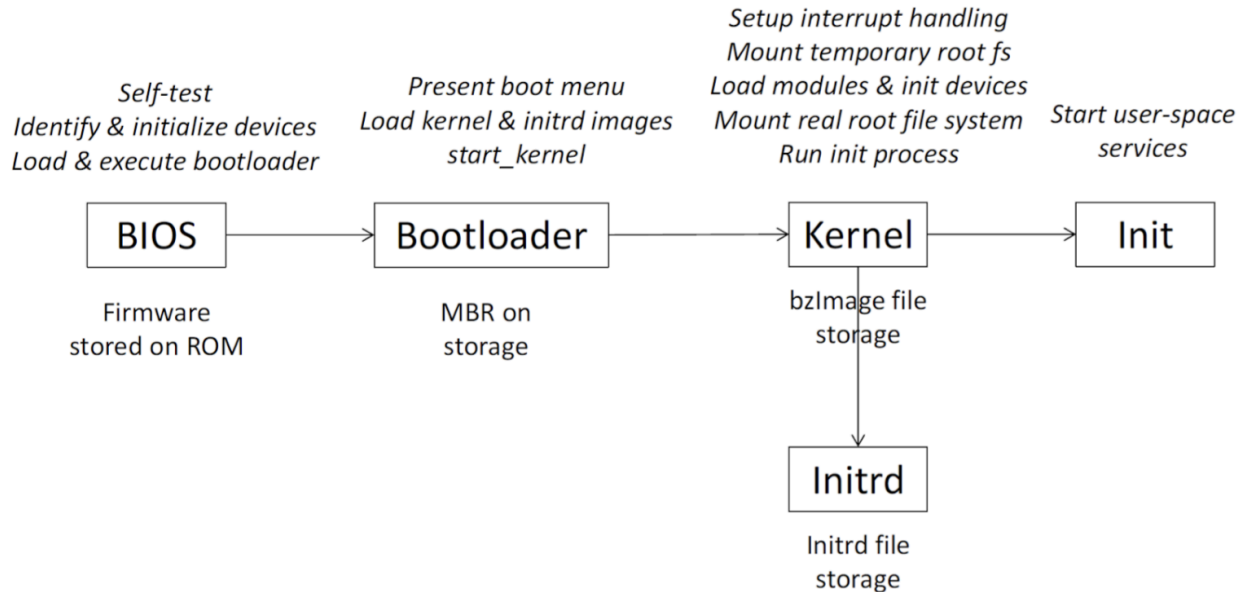
Sample Output:



```
linustorvalds@linustorvalds: ~/Desktop
linustorvalds@linustorvalds:~/Desktop$ uname -a
Linux linustorvalds 6.10.0CSE330Fall2024LinusTorvalds #1 SMP PREEMPT_DYNAMIC Tue
Aug 20 02:17:29 MST 2024 x86_64 x86_64 x86_64 GNU/Linux
linustorvalds@linustorvalds:~/Desktop$
```

References:

- Command-line tools are important for kernel development. For example, the above Step 3 requires the use of a variety of command-line tools. To help you become familiar with the command-line interface, try to do everything from a shell terminal, instead of using the GUI.
- If you don't know what command to use, the quickest way to find is through a search engine. If you don't understand the usage of a command, the easiest way is to run `man` followed by the command name.
- There are many Linux command-line references on the Web. Here is a good one:
http://wireless.ictp.it/school_2003/docs/linux/linux_guides/linux_command_reference.pdf
- Follow the principle of least privilege and do not do everything as the admin user. Use `sudo` to elevate your privilege only when necessary. Learn how to properly use `sudo`:
<https://help.ubuntu.com/community/RootSudo>
- Linux boot process



Submission Requirements & Guidelines

Project 1 is due by **September 06, 2024**. Submit your work following the below guidelines:

- **Submission:** Submit the following in a Zip file to Canvas:
 - Screenshots of the outputs.
 - You should have one screenshot of the output from step 2 and one screenshot of the output from step 3.
 - You can use VirtualBox's screenshot function to take a screenshot (to take a screenshot in VirtualBox, go to View-> Take Screenshot).
 - For UTM you can use the macOS keyboard shortcut to take a screenshot (Command + Shift + 3).
 - There **MUST** be two screenshots with the following names:
 - *lsb_release.png*
 - *uname.png*
- **Canvas submission rules:**
 - Create a .zip file with all your submission to include all the aforementioned screenshots.
 - You **MUST** name the zip file following the below naming convention. "Project-1-<Your-ASU ID>.zip" e.g. Project-1-1225754101.zip
 - Do not submit any other source code
 - Do not submit any binaries
- If your submission does not adhere to submission guidelines and naming conventions, autograding will fail and you will receive **zero** points.

Testing

- Run the following commands shown in the screenshot below in your terminal to obtain test scripts we have provided to you:

```
vboxuser@vboxuser-VirtualBox: ~/CSE330-OS
vboxuser@vboxuser-VirtualBox:~$ git clone git@github.com:visa-lab/CSE330-OS.git
Cloning into 'CSE330-OS'...
remote: Enumerating objects: 22, done.
remote: Counting objects: 100% (22/22), done.
remote: Compressing objects: 100% (19/19), done.
remote: Total 22 (delta 6), reused 15 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (22/22), 5.66 KiB | 1.42 MiB/s, done.
Resolving deltas: 100% (6/6), done.
vboxuser@vboxuser-VirtualBox:~$ cd CSE330-OS/
vboxuser@vboxuser-VirtualBox:~/CSE330-OS$ git checkout project-1
branch 'project-1' set up to track 'origin/project-1'.
Switched to a new branch 'project-1'
vboxuser@vboxuser-VirtualBox:~/CSE330-OS$ ls
README.md  test_zip_contents.sh
vboxuser@vboxuser-VirtualBox:~/CSE330-OS$
```

- Use [test_zip_contents.sh](#) to ensure your submission adheres to the file layout described in this document. If any of the screenshots are missing, the script will inform you as well as giving a summary of the correctness once it is finished running.
- If your submission fails to pass [test_zip_contents.sh](#) script check during grading you will receive **zero** points.

Grading Rubrics

Criteria	Test	Pts
Ubuntu is installed The output contains the correct Ubuntu version	<pre>lsb_release -a</pre> Expected output: <pre>No LSB modules are available. Distributor ID: Ubuntu Description: Ubuntu 24.04 LTS Release: 24.04 Codename: noble</pre>	50

New kernel is installed 1. The output contains the updated kernel version 2. Name in the output should match with the student's name. 3. Must be SMP	<pre>uname -a</pre> Check both the kernel version and SMP. Expected output: Linux vboxuser-VirtualBox 6.10.0CSE330Fall2024LinusTorvalds #1 SMP PREEMPT_DYNAMIC Tue Jul 30 18:43:02 MST 2024 x86_64 x86_64 x86_64 GNU/Linux	50
Submission Content 1. Zip file should contain only the required screenshots and source code.	<pre>./test_zip_contents.sh <path/to/zip/file.zip></pre> Checking for files - File Found: lsb_release.png - File Found: uname.png Summary ----- Got 2 out of 2 expected files	-5 pts for unwanted files

Policies

1. Late submissions will **absolutely not** be graded (unless you have verifiable proof of emergency). It is much better to submit partial work on time and get partial credit for your work than to submit late for no credit.
2. Every student needs to **work independently** on this exercise. We encourage high-level discussions among students to help each other understand the concepts and principles. However, a code-level discussion is prohibited and plagiarism will directly lead to failure of this course. We will use anti-plagiarism tools to detect violations of this policy.
3. **The use of generative AI tools is not allowed** to complete any portion of the assignment.